# SECURE CODING REVIEW REPORT

## Security Assessment and Remediation of a Python Flask Web Application

---

**Prepared For**

**CodeAlpha**

**Cyber Security Internship**

---

**Task**

**Secure Coding Review**

---

**Prepared By**

**Sovon Mitro**

**Cyber Security Intern**

---

**Programming Language:** Python

**Framework:** Flask

**Database:** SQLite

**Static Analysis Tool:** Bandit

---

**Submission Date**

10 August 2026

# 1. Executive Summary

This report presents a secure coding review of a small web application developed using Python and the Flask web framework. The application provides basic user registration and login functionality and uses an SQLite database for storing user information.

The objective of the review was to identify security vulnerabilities through manual source-code inspection, manual security testing, and automated static analysis. After identifying the vulnerabilities, appropriate secure coding practices were applied to remediate them. The application was then tested again to verify that the vulnerabilities had been successfully addressed.

The initial security review identified five security issues:

1. Reflected Cross-Site Scripting (XSS)
2. Password disclosure
3. Flask debug mode enabled
4. SQL injection
5. Plaintext password storage

The Bandit static analysis tool initially identified three security findings. These consisted of one high-severity finding related to Flask debug mode and two medium-severity findings related to unsafe SQL query construction.

The identified vulnerabilities were remediated by implementing Jinja2 HTML escaping, removing sensitive password information from application responses, disabling Flask debug mode, using parameterized SQL queries, and implementing secure password hashing.

After remediation, the application was retested and a final Bandit scan was performed. The final scan reported **“No issues identified”**, with zero high-, medium-, and low-severity findings.

# 2. Introduction

Secure coding is an important part of software development because insecure programming practices can introduce vulnerabilities that attackers may exploit to access sensitive information, manipulate application behavior, or compromise systems.

A secure application should not only provide the expected functionality but should also properly handle untrusted input, protect authentication information, safely interact with databases, and use secure configuration settings.

For this task, a Python Flask web application was selected for a security review. The application was designed as a small educational web application containing registration and login functionality. The application was reviewed for common security vulnerabilities and insecure coding practices.

The main objectives of this security review were to:

- Identify vulnerabilities in the application source code.
- Perform manual security testing.
- Use a static analysis tool to identify security weaknesses.
- Evaluate the potential impact of identified vulnerabilities.
- Apply appropriate secure coding practices.
- Retest the application after remediation.
- Verify the final application using automated static analysis.

# 3. Application Overview

The application reviewed in this project is a small web application developed using Python and Flask.

The application provides two primary functions:

- User registration
- User login

User information is stored in an SQLite database.

The application accepts user-controlled information such as usernames, email addresses, and passwords. Because these inputs are controlled by users, they were specifically examined for potential security problems.

The project contains two versions of the application:

**Vulnerable Application:** The original version containing insecure coding practices used for testing and vulnerability identification.

**Secure Application:** The remediated version containing the security fixes identified during the review.

The vulnerable version was retained so that the original security weaknesses could be compared with the secure implementation.

# 4. Technologies and Tools Used

The following technologies and tools were used during the project.

| Technology / Tool | Purpose |
|---|---|
| Python | Programming language used to develop the application |
| Flask | Web application framework |
| SQLite | Database management |
| Werkzeug | Password hashing and verification |
| Anaconda Navigator | Python environment management |
| Spyder IDE | Application development and testing |
| Bandit | Python static security analysis |
| Web Browser | Manual application testing |

# 5. Security Review Methodology

The security assessment was performed using a combination of manual and automated security techniques.

## 5.1 Manual Source-Code Review

The application source code was manually inspected to identify potentially insecure programming practices.

The review focused on:

- User input handling
- HTML output generation
- SQL query construction
- Password storage
- Password verification
- Flask configuration
- Sensitive information disclosure
- Error handling

## 5.2 Manual Security Testing

The application was executed locally and tested using specially crafted inputs.

Testing was performed to determine whether the suspected vulnerabilities could actually be triggered during application use.

Examples included testing for Cross-Site Scripting, SQL injection, password exposure, and authentication behavior.

## 5.3 Static Analysis

Bandit was used to automatically analyze the Python source code for common security problems.

The initial Bandit scan identified three findings:

- B201 – Flask debug mode enabled
- B608 – Potential SQL injection through string-based query construction
- B608 – Potential SQL injection through string-based query construction

## 5.4 Remediation

Each identified security issue was analyzed and corrected using appropriate secure coding practices.

## 5.5 Retesting

The same security tests were performed against the remediated application to determine whether the vulnerabilities were successfully eliminated.

## 5.6 Final Verification

A final Bandit scan was performed against the secure application.

The final scan reported no security issues.

# 6. Initial Static Analysis Results

The first Bandit scan was performed against the vulnerable application.

The results were:

| Severity | Number of Issues |
|---|---|
| High | 1 |
| Medium | 2 |
| Low | 0 |

The three findings were:

### B201 – Flask Debug Mode

This finding indicated that the Flask application was configured with debug mode enabled.

### B608 – SQL Expression

This finding indicated a potential SQL injection risk caused by constructing SQL statements using string-based query construction.

The B608 finding occurred twice because two separate database operations used this insecure approach.

The initial Bandit results demonstrated that the application contained security weaknesses that required remediation.

# 7. Finding SC-01 – Reflected Cross-Site Scripting

## 7.1 Description

Cross-Site Scripting, commonly known as XSS, occurs when an application accepts untrusted user input and places it into an HTML response without appropriate escaping or encoding.

The original application directly inserted user-controlled information into HTML responses.

The username entered during registration was placed directly into the registration-success response. Because the value was not properly escaped, HTML and JavaScript supplied by the user could be interpreted by the browser.

## 7.2 Vulnerability Demonstration

To test for XSS, the following JavaScript payload was entered as the username:

**<script>alert('XSS')</script>**

When the vulnerable application processed this input, a JavaScript alert popup appeared in the browser.

This demonstrated that the application was executing user-supplied JavaScript.

The test therefore confirmed the presence of an XSS vulnerability.

## 7.3 Potential Impact

An attacker exploiting XSS may be able to:

- Execute JavaScript in another user's browser.
- Modify webpage content.
- Manipulate the victim's interaction with the application.
- Perform client-side phishing attacks.
- Access information available to client-side JavaScript.

The exact impact depends on the application's authentication, session-management, and browser security controls.

## 7.4 Remediation

The vulnerability was remediated by replacing direct insertion of user input into HTML with Jinja2 template rendering.

Jinja2's automatic HTML escaping was used so that special HTML characters supplied by a user are treated as text instead of being interpreted as HTML or JavaScript.

The application was also modified so that dynamically generated login responses use the same safe rendering approach.

## 7.5 Verification

The same XSS payload was submitted to the secure application.

Instead of executing the JavaScript, the payload was displayed as ordinary text.

No JavaScript popup appeared.

This confirmed that the original XSS vulnerability had been successfully remediated.

**Status: Fixed**

# 8. Finding SC-02 – Password Disclosure

## 8.1 Description

Passwords are sensitive authentication credentials and should never be unnecessarily displayed by an application.

During the security review, password information was identified as being handled insecurely in the original application.

Displaying authentication credentials in an application response can expose users' passwords to unauthorized individuals.

## 8.2 Security Impact

Password disclosure can result in credential compromise.

If an attacker gains access to an exposed password, the attacker may use it to authenticate to the application. The situation can be more serious if the user has reused the same password on other services.

## 8.3 Remediation

The registration response was modified so that only necessary information, such as the username and email address, is displayed.

The password is no longer included in the registration response.

This follows the principle of minimizing sensitive information exposure.

## 8.4 Verification

After the remediation, the registration process displayed the username and email address without displaying the password.

The password therefore remained confidential during the registration process.

**Status: Fixed**

# 9. Finding SC-03 – Flask Debug Mode Enabled

## 9.1 Description

Bandit identified a security issue related to Flask debug mode.

The original application was configured with `debug=True`.

Bandit reported this issue as **B201: flask_debug_true**.

The finding was classified as high severity with medium confidence.

## 9.2 Security Impact

Flask debug mode is intended for development and troubleshooting.

When exposed in an inappropriate environment, the Flask/Werkzeug debugger can reveal sensitive debugging information and may introduce serious security risks.

Debug mode should therefore not be enabled when deploying an application to a production environment.

## 9.3 Remediation

The application configuration was changed from `debug=True` to `debug=False`.

The application was then restarted and tested to ensure that it continued to function normally.

## 9.4 Verification

After disabling debug mode, the application continued to operate correctly.

A subsequent Bandit scan no longer reported the B201 finding.

**Status: Fixed**

# 10. Finding SC-04 – SQL Injection

## 10.1 Description

SQL Injection is a vulnerability that occurs when user-controlled input is directly incorporated into SQL statements.

The original application constructed SQL queries using Python f-strings.

For example, the username supplied by a user was inserted directly into the SQL statement used to retrieve user information.

The registration functionality also constructed an SQL INSERT statement using direct string interpolation.

This created a potential SQL injection vulnerability because user input was being treated as part of the SQL statement rather than strictly as data.

## 10.2 Static Analysis Finding

Bandit identified both vulnerable SQL statements as B608 findings.

The finding description indicated a possible SQL injection vector through string-based query construction.

The findings were classified as medium severity.

## 10.3 Manual Testing

Manual testing was performed using specially crafted input containing SQL-related characters.

During testing, the vulnerable application generated an SQLite database error identified as `sqlite3.OperationalError`.

This demonstrated that user-controlled input was affecting the SQL query and confirmed the security concern identified during source-code review.

## 10.4 Potential Impact

SQL injection can potentially allow attackers to:

- Bypass authentication.
- Read unauthorized database information.
- Modify database records.
- Delete database records.
- Manipulate application behavior.

The actual impact depends on the privileges of the application's database connection and the structure of the application.

## 10.5 Remediation

The vulnerable SQL queries were replaced with parameterized SQLite queries.

Instead of directly inserting user input into SQL statements, placeholders were used and the user-supplied values were passed separately to the database driver.

This ensures that user input is treated as data rather than executable SQL syntax.

Parameterized queries were implemented for both:

- User registration
- User lookup during login

## 10.6 Verification

The SQL injection test was repeated against the secure application.

The previously observed SQLite error no longer occurred, and the supplied input was handled as normal data.

The two B608 findings also disappeared from the subsequent Bandit scan.

This confirmed that the SQL injection vulnerabilities had been successfully remediated.

**Status: Fixed**

# 11. Finding SC-05 – Plaintext Password Storage

## 11.1 Description

The original application stored user passwords directly in the SQLite database.

During the review, a test user's password could be observed directly in the database.

This means that anyone who obtained unauthorized access to the database could immediately read the stored passwords.

## 11.2 Security Impact

Plaintext password storage presents a serious security risk.

If an attacker gains access to the database, there is no need to crack the passwords because the original credentials are already available.

The impact can be increased if users reuse the same password across multiple websites or applications.

## 11.3 Remediation

The application was modified to use Werkzeug's password hashing functionality.

During registration, the user's password is converted into a secure password hash before being stored in the database.

During login, the password supplied by the user is compared with the stored hash using Werkzeug's password verification function.

The original plaintext password is therefore not stored in the database.

## 11.4 Verification

After remediation, a test user's database record contained a password hash beginning with a value similar to `scrypt:32768:8:1` rather than the original plaintext password.

The correct password successfully authenticated the user, while an incorrect password was rejected.

This confirmed that password hashing and verification were functioning correctly.

**Status: Fixed**

# 12. Secure Coding Practices Applied

Several secure coding practices were applied during the remediation process.

## 12.1 Parameterized SQL Queries

SQL statements should not be constructed by directly inserting user-controlled values.

Parameterized queries were implemented so that user input is handled as data rather than SQL instructions.

## 12.2 Secure Password Hashing

Passwords should never be stored in plaintext.

The application now hashes passwords before storing them and verifies passwords against their stored hashes during authentication.

## 12.3 HTML Output Escaping

Untrusted user input should not be directly inserted into HTML.

Jinja2 template rendering with automatic HTML escaping was used to prevent user-controlled HTML and JavaScript from being executed.

## 12.4 Secure Flask Configuration

Flask debug mode was disabled.

Development debugging features should not be enabled when an application is deployed in a production environment.

## 12.5 Minimized Information Disclosure

Sensitive information, especially passwords, was removed from application responses.

Applications should only return information that is necessary for the intended operation.

## 12.6 Static Security Analysis

Bandit was used to identify security issues in the Python source code.

Static analysis can help developers identify common security problems early in the development process.

## 12.7 Manual Security Testing

Automated tools alone cannot identify every type of security vulnerability.

Manual testing was therefore performed to verify the actual behavior of the application and confirm whether suspected vulnerabilities could be triggered.

# 13. Before and After Security Comparison

The security improvements made during the project can be summarized as follows:

| Vulnerability | Initial Condition | Remediation | Final Condition |
|---|---|---|---|
| Reflected XSS | User input could execute JavaScript | Jinja2 HTML escaping | Payload displayed as text |
| Password Disclosure | Sensitive password information could be exposed | Removed password from response | Password not displayed |
| Flask Debug Mode | Debug mode enabled | Changed to `debug=False` | Debug mode disabled |
| SQL Injection | User input directly incorporated into SQL | Parameterized queries | Input treated as data |
| Plaintext Password Storage | Password stored directly | Secure password hashing | Password hash stored |

# 14. Final Static Analysis Results

After all identified vulnerabilities had been remediated, Bandit was executed again against the secure application.

The final result was:

**Test results: No issues identified.**

The final severity results were:

| Severity | Number of Issues |
|---|---|
| Undefined | 0 |
| Low | 0 |
| Medium | 0 |
| High | 0 |

The final Bandit scan analyzed 113 lines of Python code.

The comparison between the initial and final scans is shown below:

| Bandit Result | Initial Application | Secure Application |
|---|---|---|
| High | 1 | 0 |
| Medium | 2 | 0 |
| Low | 0 | 0 |
| Total | 3 | 0 |

The final result demonstrates that all three issues originally identified by Bandit were successfully addressed.

# 15. Testing Results

The application was tested before and after remediation.

| Security Test | Vulnerable Application | Secure Application |
|---|---|---|
| XSS testing | JavaScript popup appeared | Payload displayed as text |
| Password disclosure | Sensitive password information exposed | Password not displayed |
| SQL injection testing | SQLite error generated | Input treated as normal data |
| Correct password | Authentication tested | Successful authentication |
| Incorrect password | Authentication tested | Login rejected |
| Database password storage | Plaintext password | Secure password hash |
| Flask debug configuration | Debug mode enabled | Debug mode disabled |
| Bandit static analysis | 3 findings | No issues identified |

# 16. Limitations

This project was conducted on a small educational Flask application and therefore does not represent a complete production-level security audit.

The review focused primarily on the vulnerabilities introduced by the application's source code and configuration.

The following areas were not comprehensively assessed:

- Session management
- Cross-Site Request Forgery protection
- Authorization and access control
- Security headers
- HTTPS/TLS configuration
- Dependency vulnerability scanning
- Rate limiting
- Account lockout mechanisms
- Production web-server configuration
- Security logging and monitoring
- Full penetration testing

A real production application would require a much broader security assessment covering these and other areas.

# 17. Conclusion

This secure coding review demonstrated the importance of identifying and addressing security vulnerabilities during the software development process.

A Python Flask web application was selected and reviewed using manual source-code inspection, manual security testing, and automated static analysis with Bandit.

The initial review identified five security issues: reflected Cross-Site Scripting, password disclosure, Flask debug mode, SQL injection, and plaintext password storage.

The identified vulnerabilities were remediated using appropriate secure coding practices. Jinja2 automatic HTML escaping was implemented to prevent XSS, sensitive password information was removed from application responses, Flask debug mode was disabled, parameterized SQL queries were introduced, and passwords were securely hashed before being stored in the database.

The application was then retested to verify the effectiveness of the changes. The XSS payload was no longer executed, SQL-related input was treated as data, passwords were stored as hashes rather than plaintext, authentication continued to function correctly, and debug mode was disabled.

The final Bandit scan reported **“No issues identified”**, with zero high-, medium-, and low-severity findings.

The project demonstrates that secure software development requires more than simply making an application functional. Developers must consider how user input is processed, how sensitive information is stored and displayed, how databases are accessed, and how applications are configured.

Combining **manual code review, manual security testing, static analysis, remediation, and retesting** provides a stronger approach to identifying and reducing application security risks.